Язык программирования Python

Циклы

Доцент кафедры ИВТ, к.т.н.
Проскурин Александр Викторович

Циклические процессы

Вычислительный процесс называется **циклическим**, если он содержит неоднократное выполнение одной и той же последовательности действий (блока кода).

Одно выполнение блока называется **итерацией**.

В Python для этого существует два оператора: **while** и **for**.

Циклы очень полезны в программировании, поскольку это позволяет не прописывать вручную одну и ту же операцию несколько раз.

Допустим нам надо вывести на экран 5 чисел от 3 до 7. Как бы это выглядело без циклов:

```
print(3)
print(4)
print(5)
print(6)
print(7)
```


© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Оператор цикла while (1)

while – самый простой вид циклов, который выполняет вложенные инструкции (блок) до тех пор, пока условие истинно.

Предыдущий пример можно переписать так:

```
# создаем переменную, равную 3
var = 3
# прописываем цикл с условием - выполнять до
# тех пор, пока переменная меньше или равна 7
while var <= 7:
    # выводим значение переменной
    print(var)
    # увеличиваем переменную на 1
    var += 1
```

Не забывайте, что цикл когда-нибудь должен завершиться! Т.е. условие должно в итоге оказаться ложным. В данном случае нужно не забыть строку **`var += 1`**

© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Оператор цикла while (2)

Графически работу оператора цикла **while** можно выразить следующим образом:

Если забыть последнюю вложенную инструкцию, то цикл станет бесконечным (никогда не завершится).

Цикл while удобно использовать когда конкретное количество итераций неизвестно, но есть условие выполнения или останова.

© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Пример использования цикла while (1)

Приведём пример вычисления факториала. Для этого используется следующая формула:

$$n! = \prod_{i=1}^{n} i \quad \text{она означает} \quad n! = 1 \cdot 2 \cdot 3 \cdot \ldots \cdot n-1 \cdot n$$

```
n = 5   # Число n из формулы
i = 1   # Номер текущей итерации
factorial = 1   # Хранит факториал числа n (0! == 1)
while i <= n:
    factorial *= i  # Вычисляем факториал для нового i
    print('Значение ', i, '! = ', factorial)
    i += 1
```

Обратите внимание, что на каждой итерации мы получаем факториал нового числа.

```
Значение  1 ! =  1
Значение  2 ! =  2
Значение  3 ! =  6
Значение  4 ! =  24
Значение  5 ! =  120
```


© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Пример использования цикла while (2)

Необходимо найти основание натурального логарифма, которое вычисляется по следующей формуле:

$$e = \sum_{n=0}^{\infty} \frac{1}{n!}$$

Поскольку количество итераций заранее неизвестно, то вычислять данную формулу будем с заданной точностью eps:

```
e = 0    # Здесь накапливаем сумму
n = 0    # Переменная n из формулы
factorial = 1   # Хранит факториал числа n (0! == 1)
de = 1   # Хранит текущее слагаемое
eps = 0.0001    # Точность вычисления

while de >= eps:  # Вычисляем пока слагаемое больше точности
    e += de   # Добавляем слагаемое к сумме
    n += 1    # Увеличиваем на 1, иначе цикл не завершится
    factorial *= n     # Вычисляем факториал для нового n
    de = 1/factorial   # Вычисляем новое слагаемое

print('Значение e = ', e)
```

```
Значение e =  2.7182539682539684
```

© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Оператор цикла for … in

Инструкция **for … in** в Python **повторяет** указанный **блок кода для всех элементов коллекции** (списки, строки, множества и т.д.), а также любого **итерируемого выражения** (т.е. выражения, которое при повторном выполнении будет предоставлять новое значение или признак конца).

Таким образом, цикл for удобно использовать когда известно конкретное количество итераций.

Синтаксис имеет вид:

```
for переменная in итерируемое_выражение:  # заголовок
    инструкция_1
    инструкция_2
    ...
    инструкция_n
```

блок операторов

 # тело цикла

В **переменная** на каждой итерации помещается текущее значение из **итерируемое_выражение**. После этого её можно обрабатывать различными инструкциями.

© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Пример использования цикла for

Имеется строка, необходимо вывести её символы в один столбец.

```
st = 'Строчка' # Исходная строка
for symbol in st:
    print(symbol) # Вывод одного символа за раз
```

Ответ:
С
т
р
о
ч
к
а

В переменную `symbol` последовательно помещаются символы из строки `st`, которые затем выводятся на экран.

Так, на первой итерации в `symbol` будет помещён символ 'С', на второй итерации символ 'т' и т.д.

Цикл завершится как только будет достигнут конец строки.

© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Функция range()

Функция **range() создаёт последовательность из целых чисел**, которая может использоваться в качестве итерируемого выражения в цикле for.

Синтаксис использования функции:

```
range(начало, конец, шаг)
```

начало – целое число, с которого будет начинаться последовательность. Если его не указать, то по умолчанию используется 0.

конец – целое число, до которого будет создаваться последовательность. При этом само это число не войдёт в последовательность.

шаг – целое число, которое сначала прибавится к **начало**, затем к полученному значению и так до тех пор, пока не будет достигнут **конец**. Если его не указать, то по умолчанию используется 1.

© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Использование range() в цикле (1)

Если количество итераций известно заранее, то удобно использовать функцию **range()** в качестве итерируемой последовательности:

```
for i in range(1, 10, 2):
    print(i)
```

Начало

Конец

Шаг

```
Ответ:
1
3
5
7
9
```

Цикл for можно также изображать на блок-схемах, как и цикл while (через условие).

Важно помнить, что увеличение счетчика и его начальное значение заложено внутри оператора for и его не надо прописывать отдельно.

© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Использование range() в цикле (2)

Функцию range() можно применять **не указывая шаг**. В этом случае используется значение по умолчанию **равное 1**:

```
for i in range(1, 6):
    print(i)
```

Начало

Конец

```
Ответ:
1
2
3
4
5
```

Также можно **не указывать начало**. В этом случае используется значение по умолчанию **равное 0**:

```
for i in range(6):
    print(i)
```

Конец

```
Ответ:
0
1
2
3
4
5
```


© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Пример использования цикла for

Приведём пример вычисления факториала с помощью цикла for. Для этого используется следующая формула:

$$n! = \prod_{i=1}^{n} i \quad \text{она означает} \quad n! = 1 \cdot 2 \cdot 3 \cdot \ldots \cdot n-1 \cdot n$$

```
n = 5   # Число n из формулы
factorial = 1   # Хранит факториал числа n (0! == 1)
for i in range(1, n+1):
    factorial *= i  # Вычисляем факториал для нового i
    print('Значение ', i, '! = ', factorial)
```

Если сравнить полученный код с кодом для цикла while (6 слайд), то можно обнаружить, что код сократился на 2 строки – убралось объявление перед циклом переменной i и её увеличение в конце цикла.

```
Значение  1 ! =  1
Значение  2 ! =  2
Значение  3 ! =  6
Значение  4 ! =  24
Значение  5 ! =  120
```


© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Дополнительные инструкции циклов

В циклах for и while можно использовать дополнительные операторы:

- **break** (прерывает выполнение цикла);
- **continue** (прерывает выполнение текущей итерации).

В большинстве случаев их помещают внутри оператора if. Синтаксис использования аналогичен таковому в языке C/C++.

Также совместно с оператором break может использоваться конструкция **else**, которая выполняется если цикл не был прерван через break.

© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Дополнительный оператор continue

Оператор continue позволяет начать следующий проход цикла, минуя оставшиеся инструкции:

```
for var in 'Python':
    if var == 'h':
        continue
    print(var)
```

```
Ответ:
P
y
t
o
n
```

В этом примере мы перебираем последовательность символов и когда наша переменная хранит в себе символ 'h' мы используем оператор continue чтобы пропустить дальнейшую инструкцию print(var).

После этого оставшиеся итерации цикла выполняются без каких-либо отклонений (поскольку символ 'h' больше не встречается).

© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Дополнительный оператор break

Оператор break досрочно прерывает цикл:

```
var = 'Python'
for i in var:
    if i == 'h':
        break
    print(i)
```

```
Ответ:
P
y
t
```

В этом примере мы также перебираем последовательность символов по одному. Когда в переменную i попадает символ 'h', срабатывает оператор break, благодаря которому выполнение цикла прерывается.

Все оставшиеся символы не будут выведены на экран. Управление перейдёт к инструкции, которая располагается сразу после цикла.

© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Дополнительный оператор else

Оператор **else** проверяет цикл на экстренный выход (**break**). Если экстренного выхода не было, т.е. оператор **break** не был выполнен, то выполняется блок инструкций вложенный в оператор **else**.

```
var = 'Python'
for i in var:
    if i == 'a':
        print('Символ а есть в слове', var)
        break
else:
    print('Символа а нет в слове', var)
```

```
При var = 'Python' ответ
будет:
Символа а нет в слове Python
```

```
При var = 'Again' ответ
будет:
Символ а есть в слове Again
```


© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Бесконечный цикл

С помощью оператора break можно организовать условно бесконечный цикл для ввода каких-либо значений и их проверки:

```
text = ''
while True:
    text == input('Введите число или "стоп": ')
    if text == 'стоп':  # если введено слово стоп...
        print('Выход из программы!')
        break   # ...то выходим из цикла
    elif text == 'привет':
        print('Здравствуйте!')
    else:
        print('Неясно')
```

Запрос на ввод значений будет продолжаться до тех пор пока пользователь не введёт слово **«стоп»** и в ветке if не сработает оператор **break**.

© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Вложенные циклы

Python позволяет вкладывать операторы циклов друг в друга. В этом случае **на одну итерацию внешнего цикла приходится полное выполнение вложенного цикла**.

Например, код создания таблицы умножения:

```
for i in range(2, 10):
    for j in range(2, 10):
        print(i, 'x', j , '=', i*j)
    print()   # выведет пустую строку
```

```
2 x 2 = 4
2 x 3 = 6
2 x 4 = 8
2 x 5 = 10
2 x 6 = 12
2 x 7 = 14
2 x 8 = 16
2 x 9 = 18

3 x 2 = 6
3 x 3 = 9
```

На первой итерации первого (внешнего) цикла переменной **i** присваивается значение 2.

После этого происходит переход ко второму (вложенному) циклу и переменной **j** последовательно присваиваются значения от 2 до 9 включительно. При этом значение переменной **i** не изменяется.

После полного выполнения вложенного цикла происходит выполнение оставшихся инструкций внешнего и переход на новую итерацию (i присваивается значение 3).

© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева